Deltris: Tetris Compiled into Weights for the Stock Qwen3.5 Architecture

tnn1t1s

November 2026

Abstract

We compile the rules of Tetris into weights for the stock `Qwen3_5ForCausalLM` class, with no training. The model has 6 Gated DeltaNet layers and 219.7 million parameters, and it loads with the stock `transformers` class and no custom code. It is not a released Qwen checkpoint and has no language ability, and it runs under a host loop rather than `generate()`. Its recurrent state holds the board, one DeltaNet head per row, plus a display head whose state matrix is the whole board. The game runs as a token protocol in which every token the model emits is a write to that memory. Exactness rests on two properties of the construction: every residual coordinate except one padding dimension is designed to be 0 or 1, and the residual norm is the same on every token, so each RMSNorm reduces to a known constant. On top of that, a few fp32 identities hold exactly, among them $\text{silu}(30)/30 = 1$, $\sigma(30) = 1$ and a decay of exactly 1. In a differential test against a reference interpreter on 10,500 seeded games (10,132,372 tokens, of which 7,355,830 are model-emitted), we observe zero token mismatches and zero mismatches between the board decoded from the recurrent state and the reference board. The smallest logit gap between the correct token and the runner-up is 29.979 against a design value of 30. These are observations from one run in fp32 on one GPU, on games of at most 7,849 tokens. Long-run stability, reduced precision and the interpretability experiments the model was built for are not yet tested.

1 Introduction

A network whose weights were written by hand, rather than learned, has a known meaning for every parameter. That makes it a test object for methods that claim to read or repair networks: the reading can be graded against the construction instead of against behaviour alone. Compiled transformers have been used this way since RASP and Tracr [5, 9]. Deltris applies the idea to an architecture whose sequence mixer is a recurrent layer, Gated DeltaNet [11], as implemented for Qwen3.5 in `transformers` 5.22.1. Qwenputer and nesformer compiled processors into the same architecture before it (Section 8).

Gated DeltaNet keeps a matrix-valued state per head and updates it with the delta rule [6, 12]. With the gates pinned, the update is an exact overwrite of one row of the state, which turns each head into an addressable memory. Qwenputer [2] used this to compile a CPU into Qwen3.5-4B, with DeltaNet state as RAM, and nesformer [1] compiled the NES's 6502 CPU into a standalone model of the same architecture. Deltris does not emulate a processor. It compiles the game's own rules as a circuit, so the network contains no instruction set and no program counter, and its state matrix, plotted, is the board.

The contributions are these:

- A compiler from a small circuit description to weights for the stock `Qwen3_5ForCausalLM` class, with memory heads in the first DeltaNet layer and Boolean logic in the SwiGLU MLPs (Section 3).
- A set of conditions under which that circuit is exact in fp32: binary residual values, a constant residual norm, and gate and decay values that round to exactly 1 (Section 3.1).
- A token protocol for Tetris in which each model-emitted token is one memory write, held to the information the model can see (Section 3.4).
- A differential test on 10,500 games with no token or board mismatch, and a free-running game with a two-line clear decoded entirely from the recurrent state (Section 5).

2 Background: the Qwen3.5 decoder layer

We take the architecture from `modeling_qwen3_5.py` in `transformers` 5.22.1; the facts below are each checked by a test in the repository (`tests/test_arch_facts.py`). A decoder layer computes

$$h \leftarrow h + \mathrm{Mixer}(\mathrm{RMSNorm}(h)), \qquad h \leftarrow h + \mathrm{MLP}(\mathrm{RMSNorm}(h)), \tag{1}$$

with $\mathrm{RMSNorm}(x) = x \,/\sqrt{\overline{x^2} + \epsilon} \cdot (1 + w)$ and the SwiGLU MLP [7] $\mathrm{MLP}(x) = W_d\,(\mathrm{silu}(W_g x) \odot W_u x)$, where $\mathrm{silu}(u) = u\,\sigma(u)$ [3]. No projection in the MLP or the mixer has a bias.

When the layer type is `linear_attention`, the mixer is Gated DeltaNet. Per token, $W_{qkv}x$ passes through a depthwise causal convolution of width 4, then SiLU, and is split into q, k and v for each head. The scalars $\beta = \sigma(w_b^\top x)$ and $g = -e^A \,\mathrm{softplus}(w_a^\top x + b_{dt})$ set the write strength and the decay. After q and k are L2-normalised and q is divided by $\sqrt{d_k}$, each head with state $S \in \mathbb{R}^{d_k \times d_v}$ computes

$$S \leftarrow e^g S, \qquad S \leftarrow S + \beta\, k\,(v - S^\top k)^\top, \qquad o = S^\top q. \tag{2}$$

The head output is $\mathrm{RMSNorm}(o) \odot w \odot \mathrm{silu}(z)$ with $z = W_z x$, followed by an output projection. Run token by token with a cache, this is the recurrent path (`torch_recurrent_gated_delta_rule`). The first token of a game, `BOS`, arrives with no recurrent state and goes through the chunked prefill path instead (`torch_chunk_gated_delta_rule`), which computes the same recurrence with triangular solves; every later token takes the recurrent path.

3 Method

3.1 Exactness in fp32

By design, every residual coordinate except one padding dimension, FILL, holds 0 or 1 on every token; in fp32 the values carry rounding error, at most 3.22e-06 in the CPU check of Section 5. A field is either one-hot over its labels or dual-rail (each bit stored as a pair $(b, 1-b)$). A constant coordinate ONE stands in for every bias, since the architecture has none. On binary inputs, five fp32 facts make the circuit exact:

- $\sigma(30)$ rounds to 1.0, so a write logit of $+30$ gives $\beta = 1$; a logit of -30 gives $\beta < 10^{-12}$.
- With $A = -40$, $e^A \approx 4 \times 10^{-18}$, and the decay e^g is exactly 1.0. The decay input w_a is zero; the per-head b_{dt} enters through a softplus and has no effect at this scale.

- The convolution kernel is $[0, 0, 0, 1]$, so only the current token reaches q, k and v, and these are driven to $\{0, 30\}$, where $\mathrm{silu}(30) = 30$ and $\mathrm{silu}(0) = 0$ in fp32.
- A one-hot vector $30\, e_i$ L2-normalises to exactly e_i.
- $\mathrm{silu}(30)/30 = 1.0$ exactly, and $|\mathrm{silu}(-30)| < 10^{-11}$.

Under these values, a write with key e_s and value v in Equation 2 sets row s of S to v and leaves every other row unchanged, and a query e_r reads row r back.

The fifth condition is the constant-norm rule. Every embedding row is padded on FILL to the same L2 norm, and every value any layer writes into the residual goes into fresh dimensions whose number of active entries is fixed. The squared residual norm after each sublayer is therefore a constant known to the compiler, the same on every token (`Program.norm2_at` in `deltris/compiler/graph.py`). RMSNorm weights are left at 0, so each RMSNorm multiplies by a known scalar, which the compiler folds into the next projection. The same reasoning covers the gated norm on memory reads: every stored value has a fixed count of active entries, so the per-head normalisation of a read is also a known constant. Reading a slot that was never written returns zero and would break this rule, so the protocol writes every slot before reading it.

3.2 MLP primitives

With $G = 30$ and binary s and x, one SwiGLU unit with gate weight G on s and up weight $1/G$ on x outputs

$$\mathrm{silu}(Gs)\,\frac{x}{G} = s \wedge x, \tag{3}$$

exactly. This gated copy gives a multiplexer bit, a two-input AND, and one row of a lookup table (the gate sums the selected one-hot labels of the inputs, offset on ONE so that only the matching combination reaches G). A threshold on an integer sum u of binary terms takes two units,

$$\frac{\mathrm{silu}(G(u - t + 1)) - \mathrm{silu}(G(u - t))}{G} = [u \geq t], \tag{4}$$

which gives AND and OR over many inputs, counting, and rule tables. NOT and bijective relabelings are free: they are a different reading of dimensions that already exist (a dual-rail field's off-rail is its negation). The primitives live in `deltris/compiler/primitives.py`; each registers a NumPy reference function, and the compiler checks the compiled model against the composed references.

3.3 Memory

All memory heads live in the mixer of layer 0, where every write is a function of the current token alone. For a head with slots $0, \ldots, n - 1$, the token's embedding carries a one-hot slot field and a value field. The compiler sets W_{qkv} so that $k = 30\, e_s$ for the slot named by the token and $v = 30\times$ the value, and sets w_b to $+60$ on the slot field and -30 on ONE, so the write logit is $+30$ when the token names a slot and -30 otherwise. The query is fixed per head ($30\, e_r$ on ONE), so the head reads slot r after the write on every token. The mixers of layers 1 to 5 have a zero output projection.

Deltris uses 25 heads with $d_k = 20$ and $d_v = 45$:

- twenty board heads $B_0, \ldots, B_{19}$, where B_i holds board row i as a dual-rail 10-cell value and is always read, so the whole board reaches the residual on every token;

- a DISPLAY head with 20 slots, written at slot r by every row write and never read (its output projection is zero), whose 20×45 state matrix is the board;
- four single-slot registers: PIECE (one-hot type, rotation, x and y; $7 + 4 + 12 + 22 = 45$ dimensions, which sets d_v), PHASE, and two row pointers T and S used by line clearing.

3.4 Token protocol

The game is a sequence of tokens. The host sends `BOS`, one of 7 `PIECE` tokens when a new piece is needed, and one of 4 actions (`LEFT`, `RIGHT`, `ROT_CW`, `NOOP`) per gravity tick. Every other token is emitted by the model, fed back as the next input, and each one writes memory: `WROW`$(r, bits)$ writes board row r (20,480 tokens), `SETP`$(type, rot, x, y, lock)$ writes PIECE (14,784), `SRC`(s) writes S (21) and `PHASE`(p) writes PHASE (5). Control tokens (`TICK_END`, `NEED_PIECE`, `GAME_OVER` and 4 `LINES`(k) tokens) write nothing. The vocabulary has 35,309 tokens. The host loop appends a host token and lets the model generate until it emits `TICK_END`, `NEED_PIECE` or `GAME_OVER`.

A reference interpreter (`deltris/ref/isa.py`) defines the protocol, and it is held to the model's information limit: its next-token function sees only the current token and the memory after that token's write, never the trace history. An interpreter allowed to see more could define a protocol that no model with this memory can run. Enforcing the limit exposed four gaps in the first protocol design, each recorded in the repository's decision log (`DECISIONS.md`, D2 to D5):

- `BOS` writes the whole initial memory in one token, since head B_i is only ever read at its own slot.
- `SETP` carries a lock bit, so that a piece that locks after a legal move is stored at its post-move position.
- The entry to the lock and clear phases resets T.
- `LINES`(k) is emitted at the start of a clear, while the post-lock board still shows which rows are full.

A tick in play is `ACT` $\rightarrow$ `SETP` $\rightarrow$ `TICK_END`. When the piece locks, the model writes each row the piece occupies, then either spawns the next piece or clears lines: `PHASE(CLEAR)` $\rightarrow$ `LINES`(k), then for each target row $t = 19, \ldots, 0$ a `SRC` token naming the next non-full source row and a `WROW`$(t, \cdot)$ copying it, then `PHASE(SPAWN)` $\rightarrow$ `NEED_PIECE`. A clear therefore rewrites the board one row per two tokens.

3.5 The Tetris circuit

`compile_deltris()` (`deltris/build.py`) expresses the next-token function of the interpreter as a circuit over the token fields and the memory reads, and schedules each primitive in the earliest MLP whose inputs are ready. The longest path sets the depth. It is the action path: read the piece, form the candidate rotation and x, look up the candidate's cell mask, AND the mask with the board rows it covers, OR the collisions, choose the new position and the lock bit, and select the output token's fields. That chain needs six MLPs, so the model has 6 layers; every other path fits inside them. The output fields drive a one-hot LM head with a logit of 30 on the correct token, and decoding is greedy.

Table 1 lists the compiled sizes. The compiler chose them; none was tuned. More than half of the parameters (118.7 million, 54%) are the embedding and the LM head over the 35,309-token vocabulary, and 0.86% of all weights are nonzero.

Table 1: Compiled model. Source: `models/deltris/config.json` and `model.safetensors`, read by `paper/derive.py`. MLP units are rows of `gate_proj` with any nonzero weight.

hidden size	1,681
layers (all `linear_attention`)	6
DeltaNet heads, d_k, d_v	25, 20, 45
intermediate size	1,860
MLP units used, layers 0–5	1,373, 1,860, 808, 47, 437, 191
vocabulary	35,309
parameters	219,692,711
nonzero parameters	1,880,041 (0.86%)

The compiler writes a configuration and a state dict and saves them with `save_pretrained`; `AutoModelForCausalLM.from_pretrained` returns a `Qwen3_5ForCausalLM` with every tensor bit-identical. One runtime detail is not covered by the stock code path. On `transformers` 5.22.1, a model whose layers are all `linear_attention` raises in the cache's sequence-length query on a cached forward unless the caller passes `position_ids` and an attention mask given as a dictionary keyed by layer type; our runner passes both.

4 Setup

Reference. `deltris/ref/tetris.py` implements the rules: a 10×20 board with row 0 at the top, seven pieces with fixed rotation bitmaps and no wall kicks, one action then gravity per tick, and a host-supplied piece sequence from a seeded 7-bag. `isa.py` implements the protocol. A test suite checks that the two agree after every tick on 10,000 random-action games and 500 games of 100 pieces played by a greedy placement policy, together with forced cases (a four-line clear, non-contiguous clears, top-out, a rotation rejected at a wall, a lock after a legal slide). Random play clears few lines (61 over 10,000 games), which is why the greedy set exists: it yields 18,348 lines including 1,348 multi-line clears.

Primitive tests. Each primitive is compiled alone into a stock model and run token by token with a cache. The tests cover all inputs of a 6-bit field for the logic primitives, every (type, rotation, x) for the lookup tables, 300 random multiplexer selections, a 421-token sequence of memory writes and reads (400 of them random), and 10^6 alternating writes to one slot fed through the stock recurrent kernel. Every test asserts a margin of at least 0.4 and a residual norm constant to 10^{-5} relative at every layer; 39 tests pass.

Differential test. The compiled model runs the same 10,000 random and 500 greedy games, with the same seeds. The interpreter's trace is fed in token by token, and at every position where the model must emit, the argmax is compared with the interpreter's token. After every token the board is decoded from the DISPLAY head's state (`cache.get_recurrent_state`) and compared with the reference board. If every check passes, the model's own greedy rollout is the fed trace, so the fed test is equal to a free run; it lets games share one batched forward pass. The test ran as one job on an NVIDIA GB10 in fp32 with TF32 disabled for matrix multiplication and cuDNN, with PyTorch 2.15.0 and `transformers` 5.22.1, and took 1518 seconds. The optional `fla` and `causal_conv1d` kernels were absent, so the pure-PyTorch delta rule and convolution ran. Per-game results are in `results/p2-games.jsonl`.

Table 2: Differential test against the reference interpreter. Source: `results/p2-games.jsonl`, aggregated by `paper/derive.py`. "Checked" counts model-emitted tokens compared with the reference; the gap is the smallest logit difference between the correct token and the next-best token over all checked positions.

games	n	tokens	checked	lines	token fail	board fail	min gap
random	10,000	6,502,442	4,607,950	61	0	0	29.9812
greedy	500	3,629,930	2,747,880	18,348	0	0	29.9792
total	10,500	10,132,372	7,355,830		0	0	29.979

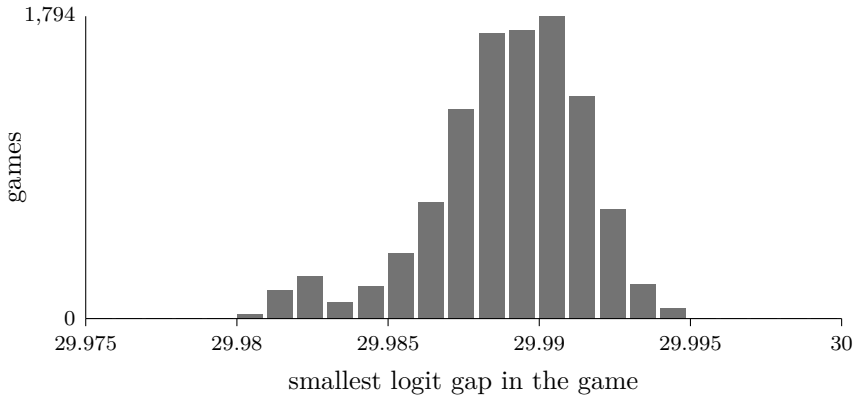


Figure 1: Smallest logit gap per game over the 10,500 games of the differential test. The design gap is 30; the smallest observed value is 29.979 and the median per-game minimum is 29.9894. Greedy decoding needs only a positive gap.

Free play. The host loop runs the model with no reference in the loop: the host sends pieces from a seeded 7-bag and actions from a greedy planner that reads only the board and piece decoded from the recurrent state, and the model emits everything else. A recorder draws each frame from the decoded state alone. The rollout used for the figures (seed 10, 1,873 tokens) is in `results/deltris-p3.pkl`.

5 Results

Table 2 gives the differential test. In this run we observe no token mismatch over 7,355,830 model-emitted tokens and no board mismatch after any of 10,132,372 tokens. The line counts in the table are summed from the fed reference trace, so they describe the test set rather than a model output; they show how many clears the token check covered. The longest game has 7,849 tokens.

Figure 1 shows how close the logits stay to the design value. The gap is set by the LM head's scale of 30 on exactly binary output fields, so any deviation measures accumulated floating-point error in the residual. The smallest per-game minimum is 29.979, a loss of about 0.02 logits. On a CPU check inside the same job, the largest residual error over 186 tokens was 3.22e-06; a one-gate probe in the primitive tests measured 1.2e-7. We have not traced the source of the remaining error. One candidate is the normalisation constants the compiler folds into projections, which are rounded to fp32; this is a hypothesis.

In free play, the 1,873-token rollout equals the interpreter's trace for the same host inputs, and after every token the board decoded from DISPLAY equals the board rebuilt from the emitted row writes. The smallest logit gap in this game is 29.983. At token 831 an I piece completes rows 17



(a) DISPLAY state, 20×45 (b) decoded board

Figure 2: Recurrent state at token 831 of a free-running game (seed 10), the `PHASE(CLEAR)` token after an I piece locks into rows 17 and 18. (a) The DISPLAY head's 20×45 state matrix, darker for larger values: row r is key slot r, and the first 20 columns hold board row r as alternating on and off rails, one pair per board column. (b) The board decoded from (a), with the two full rows outlined. The model then emits `LINES(2)` and rewrites the board bottom-up, one `SRC`/`WROW` pair per row.

and 18 (Figure 2); the model emits `LINES(2)` at token 832 and the stack shifts down two rows. The rollout took 18 seconds on the GB10. On a laptop CPU the same loop runs at about 135 ms per token, which is fast enough to play interactively in a terminal at one tick per half second.

6 Limitations

- Exactness is observed, not proved. The evidence is one run of 10,500 seeded games in fp32 on one GPU model, plus CPU tests on short games. The construction argues that every value is exact, but the logit gap of 29.979 rather than 30 shows that small errors exist, and we have not bounded their growth.
- Game length is limited. The longest tested game has 7,849 tokens. A 10^6-token game has not been run; the only million-step evidence is 10^6 writes to one slot through the recurrent kernel alone.
- Precision and kernels are fixed. Only fp32 with TF32 off was tested. bf16 is untested, and so are the optional fused kernels, which compute the same recurrence differently.
- The differential test feeds the reference trace. This equals a free run only because every check passed; a single mismatch would end the equivalence at that token. Free play was checked on one game.
- The model runs only under this repository's host loop, which passes `position_ids` and a dictionary attention mask. On `transformers` 5.22.1, `generate()` raises a `ValueError` from the cache's sequence-length query on a model whose layers are all `linear_attention`.
- The game is reduced: no wall kicks, hold, hard drop or scoring beyond a line count, and the piece sequence comes from the host. A line clear costs about 40 tokens.
- The compiled sizes were chosen by the compiler and not minimised. More than half the parameters are vocabulary.
- The runs predate the repository's first commit, so no commit hash identifies the code that produced them. They are tied to the result files listed in the reproducibility paragraph, and `results/p2-games.jsonl` can be regenerated from the seeds given there.
- The interpretability experiments that motivate the model (Section 7) have not been run.

7 Planned use

Because the meaning of every weight is known, Deltris can grade an agent that reads or repairs a network against ground truth rather than behaviour. The planned tasks give a subject the weights and configuration only. READ tasks ask what the model computes and which heads are memory, and FIX tasks plant a known corruption (for example one wrong cell in a piece's rotation table) and grade the repair against the minimal diff. The plan is to run them on four variants of increasing difficulty: the clean compile; a version with the residual stream rotated and MLP units permuted, which preserves the function exactly; the rotated version plus dense decoy circuits whose outputs never reach the logits; and a model of the same architecture trained to imitate the interpreter's traces. Rotation changes only the residual basis between modules. SiLU acts on each coordinate of q, k, v and of the MLP hidden units, so per-head state and MLP units remain readable after it, and the rotated variant tests whether a subject can trace wiring, not whether it depends on sparsity everywhere. The hypothesis behind the ladder is that current models read compiled weights mainly through basis-aligned sparsity; none of these experiments has been run, and the trained variant needs a separate grader because it has no ground-truth internals.

8 Related work

RASP describes transformer computation as a programming language, and Tracr compiles RASP programs into transformer weights to serve as ground truth for interpretability [5, 9]. Both target softmax attention. Looped transformers construct a general-purpose computer from a fixed transformer applied repeatedly [4]. Qwenputer [2] compiles an exact CPU into Qwen3.5-4B and uses Gated DeltaNet state as RAM. nesformer [1] compiles the NES's 6502 CPU, its RAM and a cartridge ROM into a standalone Qwen3.5-architecture model that loads with no custom code, so loading as a stock class is not new here; its host draws the picture from the CPU's execution log. Deltris shares the memory mechanism of both, but compiles a fixed program's rules directly. The network therefore has no instruction decoder, it is 6 layers deep, only 0.86% of its weights are nonzero, and every component is tied to one rule of the game. Its total parameter count is not small, because of the vocabulary.

Whether language models can read and write network weights directly is the question of Takehi et al. [8], whose subjects are trained networks graded on behaviour. The tasks planned in Section 7 would grade the same skills against a known construction.

The delta rule as a fast-weight update links linear attention to fast weight programmers [6]; DeltaNet parallelises it over sequence length [12], and Gated DeltaNet adds a decay gate [11]. Qwen3.5 is a successor in the Qwen model family [10] that uses Gated DeltaNet layers; we rely on its `transformers` implementation rather than a paper for the exact layer definition. Deltris pins the decay to 1 and the write strength to 1, which reduces the layer to an exact keyed overwrite.

9 Conclusion

A model of the stock Qwen3.5 architecture, with weights set entirely by a compiler, can hold a Tetris game in its recurrent state and play it token by token under a host loop. In the tests so far the compiled model agrees with the reference on every token and every board, with a logit margin within 0.03 of its design value. The next measurements are the ones listed in Section 6: a million-token game, bf16, the rotated variant, and the READ and FIX tasks.

Reproducibility. Repository: https://github.com/tnn1t1s/qwentris. Code: `deltris/` (reference in `ref/`, compiler in `compiler/`, circuit in `build.py`); dependencies pinned in `pyproject.toml` and `uv.lock` (`transformers` 5.22.1, PyTorch 2.15.0+cu132 on the GPU host). Commands: `uv run pytest -q tests/` for the reference and primitive tests; `python -m deltris.build models/deltris` to compile; `python -m deltris.difftest` for the differential test (1518 s on one GB10, fp32, TF32 off); `python -m deltris.record` for the free-play rollout (seed 10). Random games use seeds 0–9,999 and greedy games seeds 0–499. Every number in this paper is written to `paper/numbers.tex` by `paper/derive.py` from `results/p2-games.jsonl`, `results/deltris-p3.pkl`, `models/deltris`, and the job logs quoted in `PROGRESS.md`. Workaround: the runner's `position_ids` and dictionary mask (owner: this repository), to be removed when a cached forward of an all-`linear_attention` model works without them.

References

[1] dbreunig. nesformer: An nes cpu compiled into the weights of a stock qwen3.5 model. https://github.com/dbreunig/nesformer, 2026.

[2] divagr18. Qwenputer: A computer, compiled into the weights of a language model. an exact cpu inside qwen3.5-4b, with gated deltanet state as ram. https://github.com/divagr18/qwenputer, 2026.

[3] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning, 2017. URL https://arxiv.org/abs/1702.03118.

[4] Angeliki Giannou, Shashank Rajput, Jy yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped Transformers as Programmable Computers, 2023. URL https://arxiv.org/abs/2301.13196.

[5] David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Thomas McGrath, and Vladimir Mikulik. Tracr: Compiled Transformers as a Laboratory for Interpretability, 2023. URL https://arxiv.org/abs/2301.05062.

[6] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear Transformers Are Secretly Fast Weight Programmers, 2021. URL https://arxiv.org/abs/2102.11174.

[7] Noam Shazeer. GLU Variants Improve Transformer, 2020. URL https://arxiv.org/abs/2002.05202.

[8] Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of Weight-Space Programming: LLMs can natively read & write neural network weights. https://rikiyat.github.io/blog/weight-space-programming/, 2026.

[9] Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking Like Transformers, 2021. URL https://arxiv.org/abs/2106.06981.

[10] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang,

Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report, 2025. URL https://arxiv.org/abs/2505.09388.

[11] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated Delta Networks: Improving Mamba2 with Delta Rule, 2024. URL https://arxiv.org/abs/2412.06464.

[12] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing Linear Transformers with the Delta Rule over Sequence Length, 2024. URL https://arxiv.org/abs/2406.06484.